

2026 ICCAD Contest

Early Floorplanning with Global Route

聯發科技 (Mediatek)

Version: 2026-03-11

1. Introduction

IC 開案，是一個多團隊協作的高度系統性工程。每一個晶片開發，都是由市場需求、技術可行性、資源供應、產品定位等多層面的討論與規劃展開。其過程一開始由 Market team 基於市場與客戶需求，規劃出產品的目標功能、定位與初步規格 (Spec)。這份產品規格不僅包含基本運算能力、接口標準 (Interface Standard)、功耗 (Power Consumption)、封裝 (Package)等資訊，還需要考慮競爭產品的分析，以及因應未來產業趨勢，提前考量產品的長期銷售與發展方向，提升產品在未來市場中的競爭優勢。

接著，SSA (System Solution Architect)以及早期設計規劃團隊，基於這份 spec 展開全盤的產品技術評估。這個階段的評估重點在於，能否落實規格中要求的所有功能、面積、性能與成本。但由於此時尚未具備完整 RTL 或 Netlist 等細節資料，具體電路與資源統計都還未產生，無法以相當準確的實際狀況執行成本評估。開案初期，設計者通常僅掌握各 IP (Intellectual Property) 的預估面積 (Estimated Area)、佈局限制 (Shape Constraint)、以及其接口與通道需求 (Interface and Bus Requirements)。

在這個非常早期的階段，「Early Floorplanning」是一項至關重要的技術。它不僅為整個 IC 設計流程奠定基礎，並主導了晶片布局 (Chip Layout)、面積預估、互連策略 (Interconnection Planning)，以及技術可行性 (Technical Feasibility Estimation) 的初步判斷。Early Floorplanning 的主要目的是根據手上的 spec 資訊，包括 IP 的 estimated area、shape constraint、以及其 interface and bus requirements，以及各種訊號連線限制，預先規劃每個 IP block 最適合的布局、signal routing path... 等。Early Floorplanning 能有效協助團隊評測 chip area 是否符合目標、辨識潛在的佈局瓶頸 (如過度擁擠、area 超出預期、power spec 不符規格、溫度熱區問題)，可立刻進行 spec 修正或者功能調整，甚至預先發現可能導致後續設計失敗的技術風險。不但能為規格調整與專案啟動提供即時回饋，還能協助跨部門溝通，包括市場規劃、架構設計與實體設計團隊。良好的 Early Floorplanning 能大幅降低後期因 layout 異動而產生的風險與成本，並有助於長期維持產品競爭力和技術領先。

Early Floorplanning 同時提供給 market team 與 SSA 更精準的決策依據，推動專案 kick-off。良好的 Early floorplanning，不但可精細分配 chip outline 與 block 位置，還能預估訊號 routing 成本（如 wirelength、channel 等），給予後續實體開發一個最有力的 area baseline。

2. Problem Statement

本題聚焦於在 IC 專案早期規劃階段，如何根據已知規格與設計限制，實作出「Early Floorplanning with Global Route」，完成晶片初步布局與訊號連線規劃，並優化繞線長度(Wire Length)，晶片面積 (Chip Area) 與 晶片成本 (Chip Cost)。

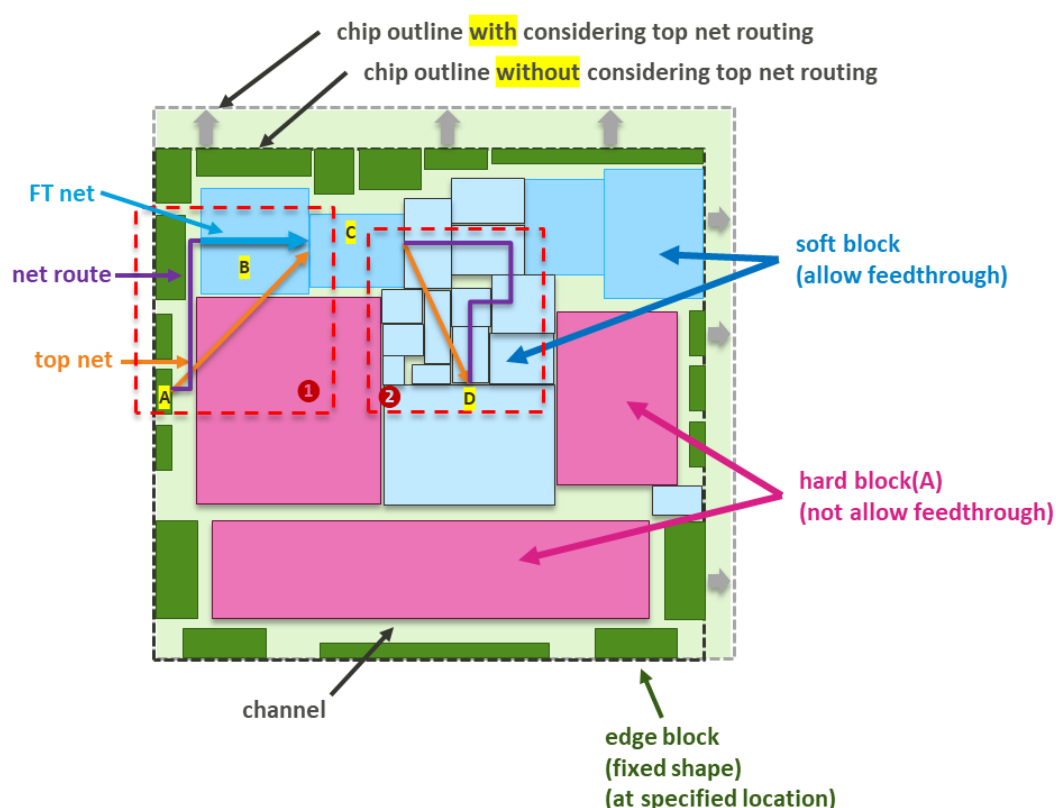


Figure 1. Early Floorplanning Layout 與 Block type

Floorplan Illustration and Definition of Terms

Figure 1. 示意 IC Early Floorplanning 的區塊類型(Block Types)、繞線路徑(Routing Paths) 與設計限制(Design Constraints)。整晶片 layout 以虛線定義的「chip outline」為邊界，所有元件必須被完整放置在此範圍內。圖中標註了 chip outline without top wire area 與 chip outline with top wire area，兩者差異在於，chip outline 在進行 top net routing 時，會因繞線而使 layout 面積擴大。。

示意圖中標示的 (1) 紅色虛線框中，橘色為示意 design 中的 logic connection, 從左下 edge block A 至中上方的 soft block C。實際走線方式可以如圖中所標示，部分走 channel，即紫色區段，經由淺綠色區塊，連接到上方藍色 soft block B 的左側。再經由 soft block B，即 feedthrough，連接到終點站的 soft block C。另一個範例，在示意圖中標示的 (2) 紅色虛線框中，connection 為 soft block C 到 soft block D。此範例為 detour route，即非走最短繞線路徑。該走線方式合法，但因 detour (走遠路)，因走線所需付出的面積相對增加。

以下為名詞定義說明：

Connection Flyline (橘色箭頭)：標出兩條訊號網路的連接實例，展示其可能經由 channel 或 feedthrough 進行。

Top net route (紫色接線)：訊號實際繞線路徑

Feedthrough (藍色接線)：於 IC 設計中，某一區塊(Block)內允許外部訊號線穿越。當訊號可以穿越 block 時，連線可以不必繞外部通道(Channel)，而是直接從區塊的一邊(Edge)進入、穿越內部後從另一邊出來。這種設計方式有助於縮短訊號路徑、減少線長並提升佈局彈性。然而，當 block 允許 feedthrough 時，通常也會伴隨額外的面積成本，需要根據實際限制進行調整。允許 feedthrough 的 block(如 soft block)，能有效減少 channel 擁堵，提升總體晶片規劃效率；不允許 feedthrough 的 block(如 hard block、edge block)，則限制訊號只能經由外部 channel。Feedthrough 的數量計算為 input/output port 數量除以 2，若有同一條 net 進出相同 block N 次，feedthrough 數量 count #N。

舉個範例 Figure 2，一條 net n_1 ，起點為 BLK01，終點為 BLK02，原始 logic view 的接線為 BLK01 的 function output port 連接至 BLK02 的 Function input port。假設經過演算法規畫後，實際路線為 (function output port A) BLK01 → channel CH1 → (Input, FT Input port A) BLK02 → BLK02 內部 → (Output, FT output port A) BLK02 → 經 channel CH2 → (Input, FT Input port B) BLK02 → BLK02 內部 → (Output, FT output port B) BLK02 → channel CH1 → (Input, Function input port A) BLK02。依據上述狀況，feedthrough count 為#2。上述例子為舉例如何計算 FT 數量，實際 net 路線，還是依照參賽者的演算法規畫。不論經由 channel 或 BLK，起點一定會經由規劃的路徑，將資料送抵終點。

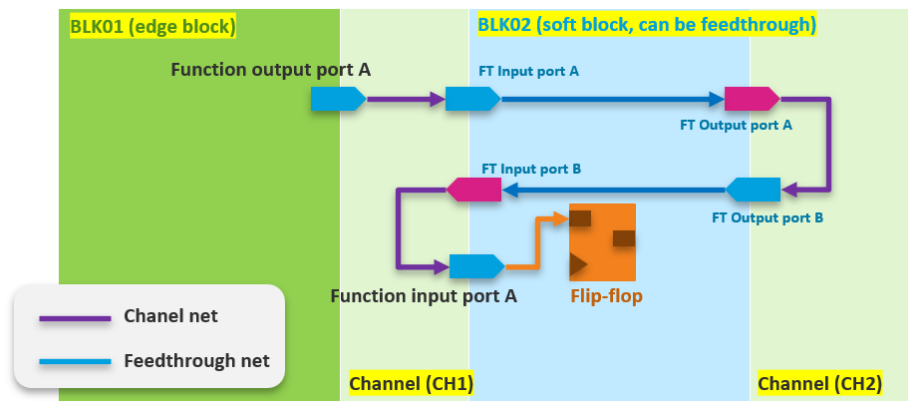


Figure 2. Path and how to calculation FT

Channel (淺綠色)：參照 Figure 5, 為各 block 間預留的線路通道，所有訊號連線若非穿越 soft block，繞線皆須走 channel。Channel wire density limit 為 25 nets/um，不論走向為垂直或水平，單方向走向在 1um 的寬度內，只允許走 25 條線。以 Figure 5 為例，Edge A 至 soft block B，最少需要的寬度為 $\#2000/\#25$, 80um 寬度的 channel。

Soft Block (藍色區塊)：可調整面積與形狀，允許訊號 feedthrough，提升彈性但會依 feedthrough 數量而額外增加其 area。Soft Block 可調整的長寬比，描述於 input file 中的 "ASPECT RATIO RANGE"。

Hard Block (粉紅色區塊)：形狀與 area 固定，不允許任何 feedthrough，所有訊號在繞線時需繞經其他路徑(如 channel)，容易成為 floorplan 與 routing 的瓶頸。

Edge Block (綠色且貼邊的區塊)：這類區塊通常承擔晶片與外部環境的介面連結功能，例如 I/O 介面模組、對外 bonding pad 或 analog 訊號端等。這些區塊的設計需考慮封裝(Package)引腳位置及走線，以減少外部連接的線路長度和訊號干擾，同時確保與 package 層的訊號接合。Edge block 本身形狀固定，不允許訊號 feedthrough，也僅允許極小範圍的移動，如 Figure 3，是確保晶片與封裝、外部電路順利整合的關鍵模組。Edge Block 需要貼齊 chip layout 邊界。

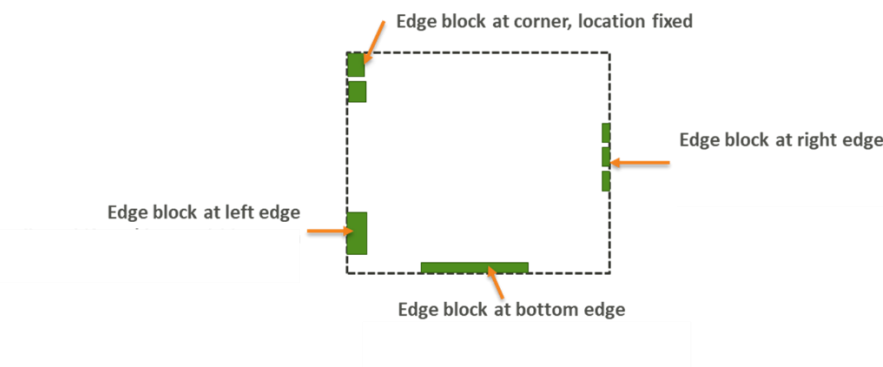


Figure 3. Edge Block

Edge block 必須根據 input csv 檔案中的 constraint 規定，擺放在指定的 Edge 邊界上。如 Figure 4 所示，整個 layout 的邊界共分為 12 個區域，分別定義為 TL、TM、TR、BL、BM、BR、LT、LM、LB、RT、RM 及 RB。當 csv 檔案中有指定某一 Edge block 的 location (例如 TL)，則該 Edge block 必須放置在圖中 TL 這個邊界上。如 Figure 4。

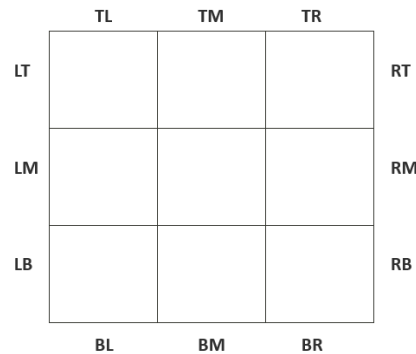


Figure 4. Constraint of Edge Block

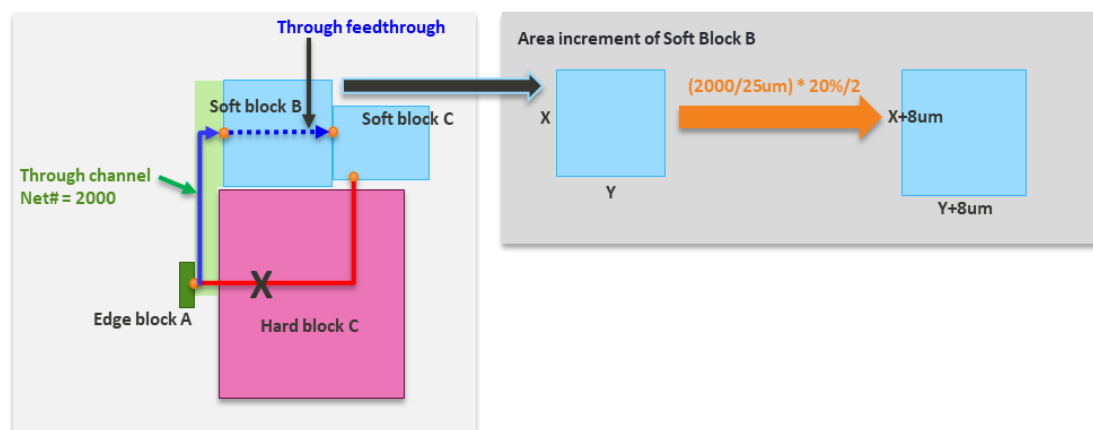


Figure 5. Example of Net Routing, partial channel and partial FT

以 Figure 5 為範例說明，當 soft block B 允許 net feedthrough 時，其面積會隨之增加。假設從 Edge block A 到 Soft block C 的連線數量為 2000，這些連線可經過 channel 再穿越 soft block B 進入 Soft block C。另一條路徑，若需經 Hard block C 通往 Soft block C，則屬於不可行路徑。

根據 feedthrough 轉換效率表 (參照 Figure 9)，對應的轉換率為 20%，而 top channel 每 1 um 可容納 25 條線。依據規則，面積的增量可計算為： $(2000 / 25 \text{ um}) \times 20\% / 2$ ，即表示 soft block B 的每個邊 (寬與高) 需額外增加 8 um。計算方式為，先取得 soft block B 現有面積的平方根，得到原本的邊長，再分別加上因 FT 所需增加的長度，最後兩者相乘，即得出調整後的新面積。

Early Floorplanning 需透過上述，做早期 floorplanning 資源配置。如何將各類 block 的特性與限制考量進去，以達成最佳面積規劃、完整的信號連接、以及所有規格條件的全面滿足，是 Early Floorplanning 的挑戰。

所有的 channel 都以「垂直」方式切割，從 outline 的左側一路到右側。每當遇到不同的 x 座標，就形成一個新的 channel 矩形區塊。範例請參考 Figure 13 channel define。Channel 計算方式如下：

- $X = \{x_0, x_1, \dots, x_M\}$
- $Y = \{y_0, y_1, \dots, y_N\}$
- 在每個 $[x_k, x_{k+1}]$ 上，block 覆蓋的區間為所有 $(y_i, y_i + h_i)$ ，且 $x_k < x_i + w_i$ 且 $x_{k+1} > x_i$ (block 覆蓋到該垂直帶)

1. 收集所有 block 的左右邊界
 - 集合所有 block 的 x 和 $x + w$, w 為 block 的 width。並加入外框的 $X = 0$ 和 $X = W$, W 為外框 outline 的 width
 - 排序得到分割線： $X = \{X_0, X_1, \dots, X_M\}$ (由小到大， $X_0 = 0, X_M = W$)
2. 每一對相鄰分割線形成一條「垂直帶 (vertical strip)」
 - 每一條垂直帶為 $[x, x_{k+1}]$
3. 在每一垂直帶內，找到所有未被任何 block 覆蓋的區間 (上下未被 block 擋住的段)
4. channel 定義：每一垂直帶中的每一個上下未被 block 覆蓋的 rectangle，即為 channel

Wire length 的計算方式是以 edge 與 edge 交界的中心點作為依據。當連線發生在 channel 與 block edge 之間時，總共有三種不同狀況，如下圖 Figure 6 所示：

1. 直線型 (Straight)：直接連線，距離為兩個中央點的直線距離。
2. L 型 (L-shape)：連線由一個拐角分兩段，總 wire length 為兩段之和。
3. Z 型 (Z-shape)：連線由兩個拐角分三段，包含兩段等長的水平 segment，wire length 為各段總和。

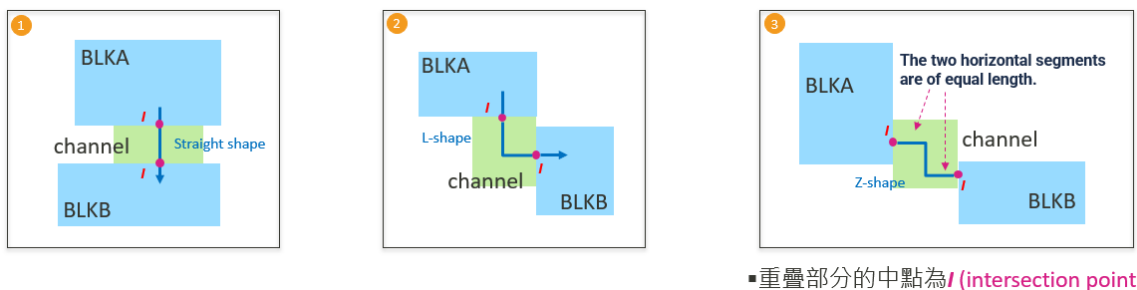


Figure 6. Path type define and wirelength calculation definition

Port Location Constraint (2026/5/25 新增)。針對不同的 block 類型 (Soft block、Hard block、Edge block)，port 位置的規範如下：

Edge block：此類型代表 chip 中 digital 包覆 analog 的 macro。為配合 chip top 的 port 連接，Edge block 必須將 port 固定於特定 edge，相關規範將於 input file 明確定義。Edge 的標示方式以順時針方向，由右側開始依序編號為 edge 1 → edge 2 → edge 3 → edge 4。

Hard block：port 需配置於單一側邊，但該側邊的選擇具備靈活性，設計者可依需求在 input file 中定義。若 edge 編號設定為 0，表示可由參賽者自由決定 macro block 的 port 位置。

Soft block：port (pin) 可彈性分布於四個 edge，不受限於單側，設計彈性更高。

3. TestCase - Input/Output/Cost Function

Input

BLOCK	AREA	WIDTH	HEIGHT	ASPECT RATIO RANGE	EDGE HARD MACRO SOFT	LOCATION	PORT EDGE	FT CONVERSION			
								<=1000	>3000, <=6000	>6000, <= 9000	>9000
BLK01	353638.5	419.5	843	1	EDGE	TL,LT	3	20%	40%	80%	100%
BLK02	409985	959	960	0.5,2	SOFT			20%	40%	80%	100%
BLK03	920640			1	MACRO		0	20%	40%	80%	100%
BLK04	399000			665	600		1	EDGE	RB,BR	1	20%
BLK05	431211	592	1198	0.5,2	SOFT			20%	40%	80%	100%
BLK06	709216			1	MACRO		0	20%	40%	80%	100%
BLK07	372949				0.5,2		SOFT		20%	40%	80%
OUTLINE	WIDTH	HEIGHT									
MAX	2916	2220									
α	0.2										
CONN MATRIX											
ON CHIP INTERFACE CONNECTION (1-1) MATRIX											
	BLK01	BLK02	BLK03	BLK04	BLK05	BLK06	BLK07				
BLK01	0	0	0	0	0	0	0				
BLK02	0	0	900	800	500	400	200				
BLK03	1200	300	0	1500	0	700	0				
BLK04	0	0	0	0	0	0	0				
BLK05	0	0	0	0	0	0	0				
BLK06	0	700	0	200	0	0	0				
BLK07	0	0	0	0	0	0	0				

Figure 7. Example of TestCase (case00)

- Definition
 - BLOCK：名稱 (如: BLK01~BLK07)。
 - AREA：block 的預估面積。
 - WIDTH、HEIGHT：部分 block 指定寬與高，其餘以面積與寬高比規範。
 - ASPECT RATIO RANGE：每個 block 允許的寬高比範圍，決定設計時形狀可調整的彈性。
 - EDGE/HARD MACRO/SOFT：block 型態分類
 - LOCATION：指定 block 的位置要求，如 BLK01 必須放置於晶片邊界。

- Definition of Feedthrough Conversion Efficiency

FT CONVERSION：每個 block 的 feedthrough 效率，依穿越 net 數量分四個檔位 (≤ 3000 , >3000 且 ≤ 6000 , >6000 且 ≤ 9000 , >9000)，對應需增加的面積百分比。

Channel wire density limit $\leq 25\text{nets}/\mu\text{m}$



e.g. wire counts = 500, require channel $\geq 20\mu\text{m}$
e.g. channel = $20\mu\text{m}$, allowed wire counts ≤ 500

Figure 8. Channel wire density limitation

Feedthrough count range	Feedthrough area conversion efficiency	Block Feedthrough addon area by width(W) & height(H) extend *channel wire density $\leq 25\text{nets}/\mu\text{m}$ ** Extend length : the calculation results are rounded up (ceil) without conditions 無條件進位 ** Area : rounded to the second decimal place 四捨五入到小數點第二位
≤ 3000	20%	e.g. Block floorplan area $87500\mu\text{m}^2$, Feedthrough wire counts = 1000 W & H, each extend = $((1000/25)*20\%)/2 = 4\mu\text{m}$. After Feedthrough, $W/H = \sqrt{87500 + 4}(\mu\text{m}) = 299.8\mu\text{m}$ Total area after including FT : $(\sqrt{87500 + 4})^2\mu\text{m}^2 = 89882.43\mu\text{m}^2 \rightarrow$ 加入FT後的最小面積, 寬高ratio限制依input file定義
$>3000, \leq 6000$	40%	e.g. Block floorplan area $87500\mu\text{m}^2$, Feedthrough wire counts = 3500 W & H, each extend = $((3500/25)*40\%)/2 = 28\mu\text{m}$. After Feedthrough, $W/H = \sqrt{87500 + 28}(\mu\text{m}) = 323.8\mu\text{m}$ Total area after including FT : $(\sqrt{87500 + 28})^2\mu\text{m}^2 = 104849.02\mu\text{m}^2 \rightarrow$ 加入FT後的最小面積, 寬高ratio限制依input file定義
$>6000, \leq 9000$	80%	e.g. Block floorplan area $87500\mu\text{m}^2$, Feedthrough wire counts = 6711 W & H, each extend = $((6711/25)*80\%)/2 = 108\mu\text{m}$. After Feedthrough, $W/H = \sqrt{87500 + 108}(\mu\text{m}) = 403.8\mu\text{m}$ Total area after including FT : $(\sqrt{87500 + 108})^2\mu\text{m}^2 = 163057.66\mu\text{m}^2 \rightarrow$ 加入FT後的最小面積, 寬高ratio限制依input file定義
>9000	100%	e.g. Block floorplan area $87500\mu\text{m}^2$, Feedthrough wire counts = 12013 W & H, each extend = $((12013/25)*100\%)/2 = 241\mu\text{m}$. After Feedthrough, $W/H = \sqrt{87500 + 241}(\mu\text{m}) = 536.8\mu\text{m}$ Total area after including FT : $(\sqrt{87500 + 241})^2\mu\text{m}^2 = 288158.52\mu\text{m}^2 \rightarrow$ 加入FT後的最小面積, 寬高ratio限制依input file定義

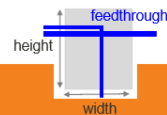


Figure 9. Feedthrough conversion efficiency table

- 連線矩陣 (Connection Matrix)

ON CHIP INTERFACE CONNECTION (1-1) MATRIX：每一對 block 間的介面連接需求。矩陣中的數字代表兩個 block 間需要多少條訊號線。

- 晶片範圍定義 (Outline)

OUTLINE：晶片可設計的尺寸範圍 (寬度與高度)，包含最大限制。

Output

- 輸出檔名：*.cfg
- 輸出格式

1. floorplan outline

預設 floorplan 的原點(左下角座標)為(0,0)，outline 在 output 格式上僅需 width 與 height 資訊。Floorplan 固定為 rectangle。

Syntax
Outline <width> < height>
Example
Outline 800 400

2. blocks shape and location

```
Syntax  
BLOCK Block_name <lx> <ly> <width> <height>  
Example  
BLOCK BLK01 0 0 300 200  
BLOCK BLK02 400 0 300 200  
BLOCK BLK03 0 200 380 200  
BLOCK BLK04 380 300 300 100
```

- Block_name 需與 input file 中的 'BLOCK' 欄位名稱一致
- (lx, ly)為 block 原點 (左下角座標)
- Width/height 為寬高

3. channel representation

```
Syntax  
NumChannels <ChannelsCount>  
CHANNEL <Channel_name> <lx> <ly> <width> <height>  
Example  
NumChannels 4  
CHANNELCH1 300 0 100 200  
CHANNELCH2 380 200 320 100  
CHANNELCH3 700 0 100 300  
CHANNELCH4 680 300 120 100
```

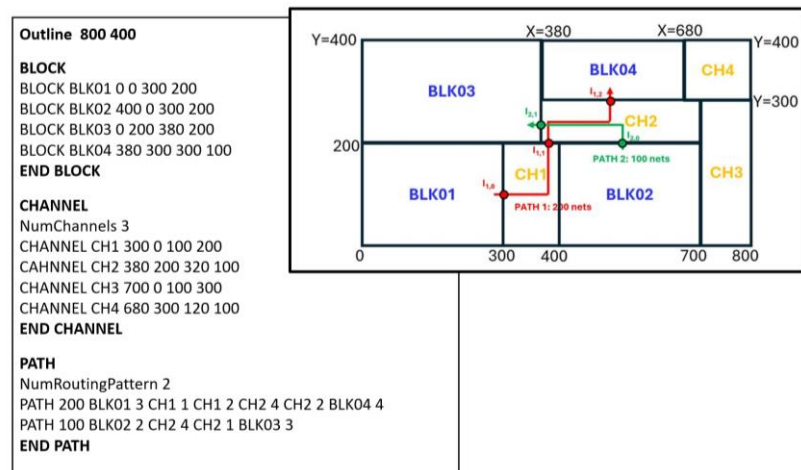
- Channel_name 名字可自行定義，字元長度不可超過 30 個
- (lx, ly)為 block 原點
- Width/height 為寬高

4. routing pattern representation

```
Syntax  
NumRoutingPattern <RoutingPatternCount>  
PATH <FT_NUM> <Block_name/Channel_name, EDGE_IDX> ...  
Example  
NumRoutingPattern 2  
PATH 200 BLK01 3 CH1 1 CH1 2 CH2 4 CH2 2 BLK04 4  
PATH 100 BLK02 2 CH2 4 CH2 1 BLK03 3
```

- 每個 block/channel rectangle 的 edge，逆時針從 1 開始編號。
左為編號 1，上為編號 2，右為編號 3，下為編號 4
- 以上圖中的範例“ PATH 200 BLK01 3 CH1 1 CH1 2 CH2 4 CH2 2 BLK04 4” 的意思為，該條 PATH 路徑中有 200 條 net，從 BLK01 出發經由 BLK01 的 edge 3 接到 CH1 的 edge 1(channel 名字為 CH1 的 rectangle)，從 CH1 的 edge2 再接到 CH2 的 edge4, 最後從 CH2 的 edge2 接到 BLK04 的 edge 4。

5. Output format Example



6. 注意事項

- 請注意，除了起始點外，所有途中經過的 rectangle，不論為 block 或 channel，皆為一進一出。
- 檢查程式將會先檢查 cfg 格式是否正確，以下狀況造成檢查程式 parser 失敗，評分結果將直接 fail
 - 無定義 outline
 - 無定義 BLOCK / CHANNEL / PATH section。每個 Section 定義項目，並以“END”為結尾
 - 非依照格式定義 “BLOCK/CHANNEL Edge”
 - PATH 無定義所含 net 數量

Cost Function

$$Cost = OutlineArea + \alpha \times TotalWireLength$$

$$TotalWireLength = \sum_{m=0}^{(\#path)-1} \sum_{n=0}^{(\#guiding_points_of_path_m)-2} HPWL(G_{m,n}, G_{m,n+1})$$

- α
 - A constant value for balancing outline area & total wirelength
 - Will be given in the input file (the value of α in each testcase may be different)
- **Overlap Segment (OS)**
 - For a path P : (BLK/channel₀, edge₀) (BLK/channel₁, edge₁) ... (BLK/channel _{i} , edge _{i})
 - We define the path has $(i + 1)/2$ overlap segments: $OS_0, OS_1, \dots, OS_{\lfloor (i+1)/2 \rfloor - 1}$
 - While OS_n represents the overlapping boundary segments of BLK/channel _{$n+2$} and BLK/channel _{$n+2+1$} in path P
- **Guiding Point (G)**
 - For a path P , it has the same number of guiding points as its overlap segments
 - We define G_n as the midpoint of OS_n
- **Total wirelength in the example**
 - $200 * (HPWL(G_{0,0}, G_{0,1}) + HPWL(G_{0,1}, G_{0,2})) + 100 * HPWL(G_{1,0}, G_{1,1})$

Suggestion Steps :

1. 計算外框面積
 - a. 外框面積 = 外框長 * 外框寬
2. 對每條路徑 (每個 net) 計算 guiding point

- a. 對每一條信號路徑 (net) , 找出它經過的 guiding point
 - b. 對於每一條網路 (path) , 按照這條 path 經過的連接順序排列 guiding points , 只有彼此直接相連的 guiding point 才列入 HPWL 計算。
3. 總線長加總
 - a. 對所有 net/路徑將 guiding point 之間的 HPWL 加總
 4. 加上 alpha 權重 , 得出 cost

Figure10 左上圖 , 展示了兩條信號路徑網路 (path 0 和 path 1) , 分別標註紅色與綠色線。path 0 (紅色、200 nets) 由 BLK01 經過 CH1 再到 BLK04。path 1 (綠色、100 nets) 由 BLK02 經過 CH2 再到 BLK04。每條路徑都是在佈局中跨越 channel 與 block 的連線。右上圖 , 展示了每條路徑實際經過的「重疊片段」 (overlap segment, OS)。每一個 channel (CH1、CH2) 的重疊區段用 OS 標註 (如 $OS_{0,0}$ 、 $OS_{0,1}$...)。這些重疊區段是將 channel/edge 界面分拆、用來指定導線的經過位置。下圖顯示各條路徑計算中的 guiding points (每個重疊片段的中點)。綠色、紅色 guiding point 分別對應 path 1、path 0。計算導線長度時 , 每個 guiding point 之間的曼哈頓距離 (HPWL) 就是路徑的導線長。

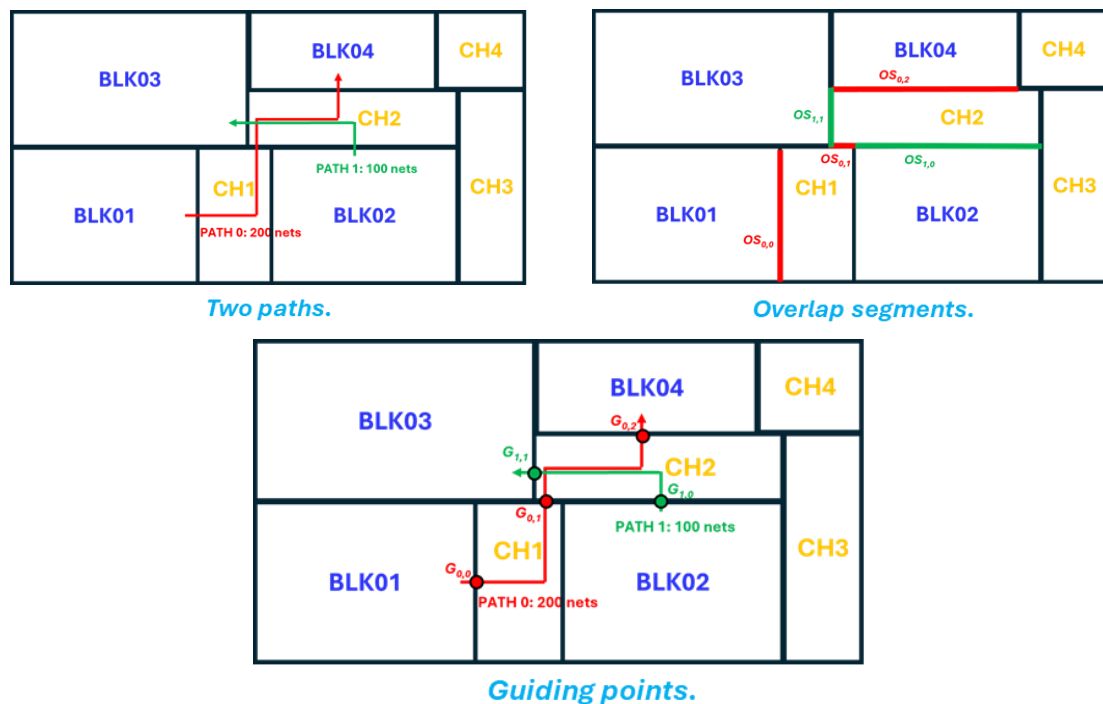


Figure 10. Initial Floorplan

Example

以 testcase00 為例 , Figure 11 是根據 input file 中提供的大小和座標所繪製的示意圖。圖中 , 綠色區塊代表 Edge block , 粉紅色區塊代表 Macro block , 藍色區塊則是 Soft

block，以上所有 block 的定義皆來自 input file。連線部分，藍色連線表示 BLK03 與其他 BLK 的连接，橘色連線表示 BLK02 與其他 BLK 的连接，紫色連線則是 BLK06 與其他 BLK 的连接。Figure 12 則統計並列出了各 connection 的 net 數量。

由於目前處於 Early floorplan 階段，尚未規劃到單條 net 的點對點詳細連線，因此此處以 block (BLK) 為單位，定義 BLK 到 BLK 的連線數量。Figure 12 右側為 input file 中根據 connection 關係所定義的 connection matrix。後續所有 channel 和 FT area 的計算都將依據這份 connection matrix 進行。

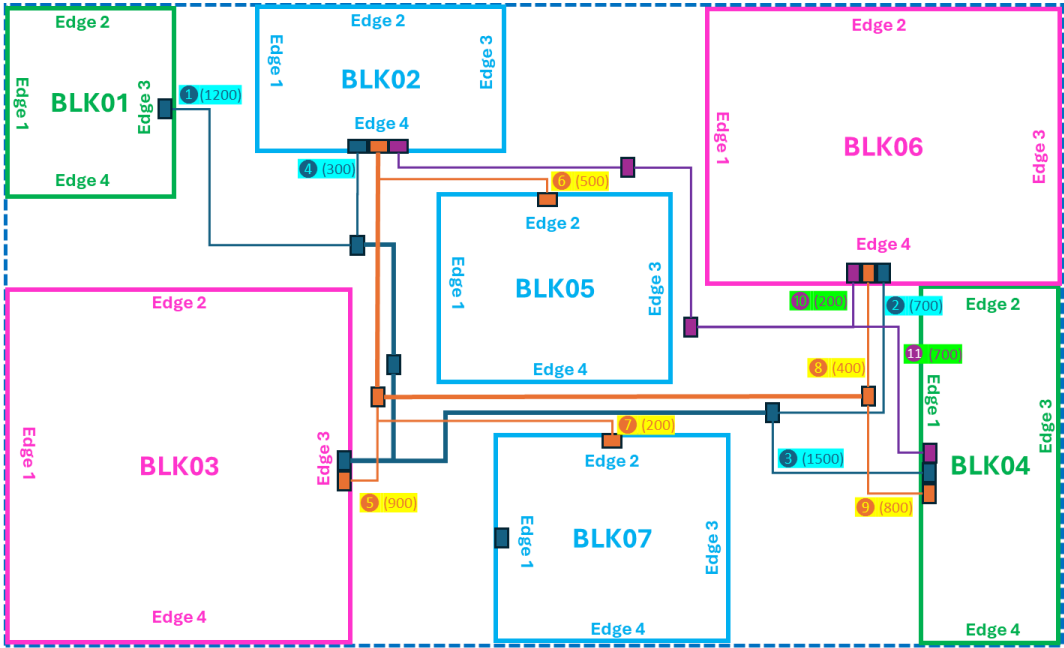


Figure 11. Initial Floorplan

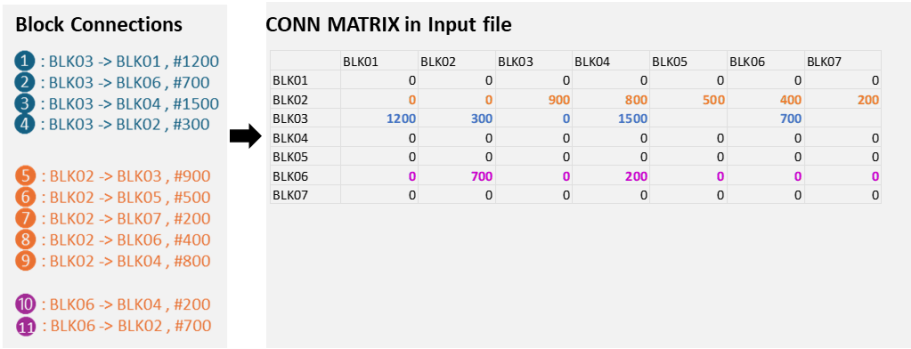


Figure 12. Block Connection and Connection Matrix

將 channel 以縱切分割成多個矩形區塊，每個區塊依序命名為 CH* (*為數字)。以 CH7 為例，經過 CH7 的連線包括：BLK03 到 BLK06 的 700 條、BLK03 到 BLK04 的 1,500 條、BLK02 到 BLK05 的 400 條、BLK02 到 BLK07 的 200 條，以及 BLK02 到 BLK04 的 800 條，合計共 3,600 條。Figure 13 說明了最基本的兩種線路配置情境。

首先，假設 BLK07 為正方形，且其面積根據 testcase00 的 input file 定義為 372,949 μm^2 ，因此預設單邊長度為該面積之平方根，即約 610.7 μm 。

第一種可能性是所有 net 均經過 channel。經過 CH7 的總 net 為 700 + 1,500 + 400 + 200 + 800 = 3,600 條 net。根據題目定義，1 μm 最大 net 數量容納率為 25 條 net，因此 channel 所需最小高度為 $3,600 / 25 = 144 \mu\text{m}$ 。CH7 區塊所需的面積為 $610.7 \mu\text{m} \times 144.0 \mu\text{m} = 87,940.8 \mu\text{m}^2$ 。

第二種可能性為所有 net 均採 FT 穿入 BLK07。依根據 FT conversion rate table，net 數量介於 3,000 到 6,000，FT 轉換率為 40%。channel 高度增量為 $(3,600 / 25) \times 40\% = 57.6 \mu\text{m}$ 。若將增長平均分配到長與寬，各邊需增加 28.8 μm 。BLK07 新長度為 $610.7 + 28.8 = 639.5 \mu\text{m}$ ，更新後面積為 $639.5 \times 639.5 = 409,012 \mu\text{m}^2$ ，相較原始面積 372,949 μm^2 ，增加了 36,063 μm^2 。

綜合比較，採用 FT 方式所需增加的面積僅 36,063 μm^2 ，顯著小於 channel routing 的面積 87,940.8 μm^2 ，提升設計效率並節省面積成本。

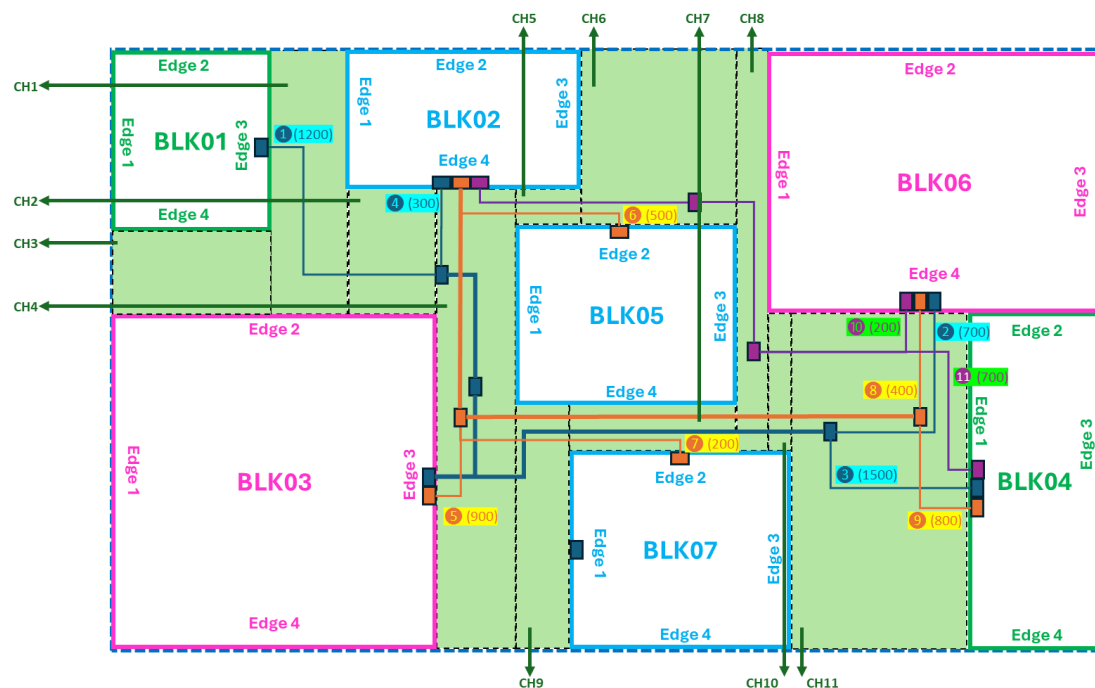


Figure 13. Defined Channel

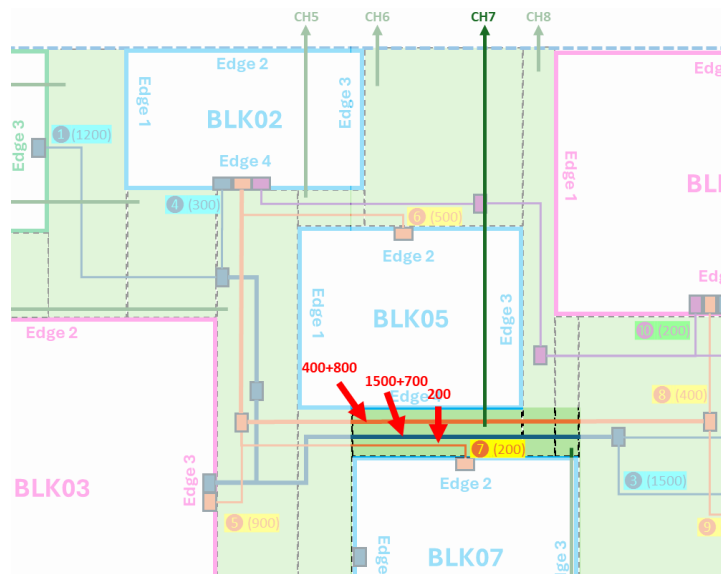


Figure 14. Example of channel / FT area calculation

[Note : The maximum net capacity per 1um is 25 nets]

Option1 : All nets remain in channel CH7

>> Minimum required height for the channel
is $(400+800+1500+700+200) / 25 = 144\mu\text{m}$
>> Assume width of BLK07 edge2 is $610\mu\text{m}$
>> total area need $144 * 610 = 87840 \mu\text{m}^2$

Option2 : All nets in channel CH7 go through FT, though BLK07

>> According to the FT conversion rate table in input file
>> net number ($>3000, \leq 6000$), Feedthrough conversion rate is 40%
>> Length of one side is $\sqrt{\text{area}}$, which is approximately $610.7 \mu\text{m}$
>> according FT conversion calculation formulate
 $(400+800+1500+700+200) / 25 * 40\% = 57.6$
>> each width and height need increase $57.6/2 \mu\text{m}$ for FT area
>> The area is updated to $(610.7+28.8)^2$ after adding FT
>> The updated area is $409012 \mu\text{m}^2$
>> Relative to the original area, $36063 \mu\text{m}^2$ area has been added.

Figure 15. Step-by-step explanation of the calculation

4. Evaluation Objectives and ICCAD Participant Guidelines

- 所有 block 必須合法排列於 chip outline 限制內、且不得重疊。
- 所有 block 均僅允許矩形。
- 優化 half-perimeter wirelength (HPWL) , 越短越好。
- 完成所有 on-chip interconnection 的 Global routing (不可有 open) 。
- Global routing 可經由 channel 或允許 feedthrough 的 block (需符合同時 feedthrough/通道限制) 完成。
- Floorplan 須符合所有 constraint 並使晶片面積成本最小。

- Runtime 限制, 所有 testcase 需於 2 小時內完成
 - Penalty condition :
 - Channel overflow
 - Feedthrough overflow
 - Fail condition
 - Format failed
 - Block overlap
 - Routing open
 - Outline constraint violation
-

5. Reference

- Channel Calculation reference code

```
struct Block {
    double x, y, w, h;
};

int count_vertical_channels(double W, double H, vector<Block> blocks) {
    vector<double> x_edges = {0, W};
    // 收集所有block的左右邊界
    for (const Block& b : blocks) {
        x_edges.push_back(b.x);
        x_edges.push_back(b.x + b.w);
    }
    // 排序並去重
    sort(x_edges.begin(), x_edges.end());
    x_edges.erase(unique(x_edges.begin(), x_edges.end()), x_edges.end());

    int channel_count = 0;
    // 對每一個垂直分割帶進行操作
    for (int k = 0; k < x_edges.size() - 1; ++k) {
        double x_left = x_edges[k];
        double x_right = x_edges[k+1];

        // 找出所有覆蓋此垂直帶的block
        vector<pair<double, double>> cover_y_intervals;
        for (const Block& b : blocks) {
            // 若block橫跨此垂直帶
            if (b.x < x_right && (b.x + b.w) > x_left) {
                cover_y_intervals.push_back({b.y, b.y + b.h});
            }
        }
    }
}
```

```

// 合併y方向覆蓋區間 ( interval union )
sort(cover_y_intervals.begin(), cover_y_intervals.end());
vector<pair<double, double>> merged;
for (auto& interval : cover_y_intervals) {
    if (merged.empty() || merged.back().second < interval.first) {
        merged.push_back(interval);
    } else {
        merged.back().second = max(merged.back().second, interval.second);
    }
}

// 計算未被覆蓋的空白y區間數 ( 即此垂直帶的channel數 )
double y_prev = 0;
for (auto& interval : merged) {
    if (interval.first > y_prev) {
        // 有一段空白channel
        channel_count++;
    }
    y_prev = max(y_prev, interval.second);
}
// 最後一段至H
if (y_prev < H) {
    channel_count++;
}
}
return channel_count;
}

```